

КРИПТОГРАФІЯ

КОМП'ЮТЕРНИЙ ПРАКТИКУМ №1

Експериментальна оцінка ентропії на символ джерела відкритого тексту

Студентів групи ФБ-41 Плотко Владислава та Пінчука Андрія

Мета роботи: Засвоєння понять ентропії на символ джерела та його надлишковості, вивчення та порівняння різних моделей джерела відкритого тексту для наближеного визначення ентропії, набуття практичних навичок щодо оцінки ентропії на символ джерела.

Порядок виконання роботи 1. Підготовка текст

master_margarita

19.09.2026 23:58

Текстовый докум...

1 336 КБ

```
1 import re
2 import sys
3 from pathlib import Path
4
5 # Алфавіт: 32 літери (Є -> І), Ї та Ъ окремо. пробіл - окремий символ.
6 LETTERS = "абвгдежзийклмнопрстуфхцшщъьэя"
7
8
9 def clean(text: str, keep_spaces: bool = True) -> str:
10     text = text.lower() # усі літери в нижній регістр
11     text = text.replace("ё", "е") # ё -> е
12     text = re.sub(f"^[^{LETTERS}]", " ", text) # не літери -> пробіл,
13     # послідовність символів -> один пробіл
14     text = text.strip()
15     if not keep_spaces:
16         text = text.replace(" ", "") # версія без пробілів
17     return text
18
19
20 def main() -> None:
21     if len(sys.argv) != 2:
22         sys.exit("Використання: python step1_prepare.py <файл UTF-8>")
23
24     raw = Path(sys.argv[1]).read_text(encoding="utf-8")
25
26     with_spaces = clean(raw, keep_spaces=True)
27     no_spaces = clean(raw, keep_spaces=False)
28
29     Path("text_with_spaces.txt").write_text(with_spaces, encoding="utf-8")
30     Path("text_no_spaces.txt").write_text(no_spaces, encoding="utf-8")
31
32     print(f"вихідний текст: {len(raw)} символів")
33     print(f"з пробілами: {len(with_spaces)} символів, "
34           f"різних символів: {len(set(with_spaces))}")
35     print(f"без пробілів: {len(no_spaces)} символів, "
36           f"різних символів: {len(set(no_spaces))}")
37
38
39 if __name__ == "__main__":
40     main()
```

```

PS C:\Users\WarningKids\Desktop\Учеба\3 курс\5 семестр\Крипта\1 лаба> python part1.py master_margarita.txt
вихідний текст: 755732 символів
з пробілами: 712082 символів, різних символів: 33
без пробілів: 598119 символів, різних символів: 32
PS C:\Users\WarningKids\Desktop\Учеба\3 курс\5 семестр\Крипта\1 лаба>

```

Правила попередньої обробки тексту, реалізовані у програмі:

1. Усі літери тексту приводяться до нижнього регістру (`str.lower()`).
2. Літера «ё» замінюється на «е».
3. Усі символи, що не належать до заданого алфавіту (розділові знаки, цифри, латиниця, символи форматування тощо), замінюються на пробіл за допомогою регулярного виразу.
4. Послідовності з кількох пробілів та інших неалфавітних символів підряд трактуються як один пробіл (замінюються одним пробілом).
5. З початку та кінця тексту видаляються зайві пробіли (`str.strip()`).
6. Для обчислень без пробілів з нормалізованого тексту додатково вилучаються всі пробіли повністю.

2. Розрахунок статистичних характеристик тексту

```

1  import csv
2  from collections import Counter
3  from math import log2
4  from pathlib import Path
5
6  OUT_DIR = Path("results")
7
8
9  def letter_counts(text: str) -> Counter:
10     return Counter(text)
11
12
13  def bigram_counts(text: str, overlapping: bool) -> Counter:
14     """Біграми зі зсувом на 1 символ (перетинні) або на 2 (неперетинні)."""
15     step = 1 if overlapping else 2
16     return Counter(text[i:i + 2] for i in range(0, len(text) - 1, step))
17
18
19  def entropy(counts: Counter) -> float:
20     """H = -sum(p * log2 p)."""
21     total = sum(counts.values())
22     return -sum((c / total) * log2(c / total) for c in counts.values())
23
24
25  def h1(text: str) -> float:
26     return entropy(letter_counts(text))
27
28
29  def h2(text: str, overlapping: bool) -> float:
30     """Ентропія біграм, поділена на 2 - потрібна ентропія НА СИМВОЛ."""
31     return entropy(bigram_counts(text, overlapping)) / 2
32
33
34  def top(counts: Counter, n: int = 5) -> str:
35     total = sum(counts.values())
36     return ", ".join(f"'{k}' {v / total:.4f}" for k, v in counts.most_common(n))
37

```

```

39 def save_table(path: Path, counts: Counter) -> None:
40     total = sum(counts.values())
41     with open(path, "w", newline="", encoding="utf-8") as f:
42         w = csv.writer(f)
43         w.writerow(["символ", "кількість", "частота"])
44         for k, v in counts.most_common():
45             w.writerow([k, v, f"{v / total:.6f}"])
46
47
48 def analyze(name: str, text: str) -> None:
49     print(f"\n=== {name} ===")
50     print(f"довжина: {len(text)}, різних символів: {len(set(text))}")
51
52     lc = letter_counts(text)
53     print(f"H1 = {entropy(lc):.4f} біт")
54     print(f" топ-5 символів: {top(lc)}")
55     save_table(OUT_DIR / f"{name}_letters.csv", lc)
56
57     for overlapping, label, tag in ((True, "перетинні", "overlap"),
58                                     (False, "неперетинні", "nonoverlap")):
59         bc = bigram_counts(text, overlapping)
60         print(f"H2 ({label}) = {h2(text, overlapping):.4f} біт "
61               f"[біграм: {sum(bc.values())}, різних: {len(bc)}]")
62         print(f" топ-5 біграм: {top(bc)}")
63         save_table(OUT_DIR / f"{name}_bigrams_{tag}.csv", bc)
64
65
66 def main() -> None:
67     OUT_DIR.mkdir(exist_ok=True)
68     for name, fname in (("with_spaces", "text_with_spaces.txt"),
69                         ("no_spaces", "text_no_spaces.txt")):
70         text = Path(fname).read_text(encoding="utf-8")
71         analyze(name, text)
72
73
74 if __name__ == "__main__":
75     main()

```

```

(base) PS C:\Users\WarningKids\Desktop\Учеба\3 курс\5 семестр\Крипта\1 лаба> python part2.py

=== with_spaces ===
довжина: 712082, різних символів: 33
H1 = 4.3748 біт
 топ-5 символів: ' ' 0.1600, 'о' 0.0929, 'а' 0.0726, 'е' 0.0683, 'и' 0.0575
H2 (перетинні) = 3.9833 біт [біграм: 712081, різних: 756]
 топ-5 біграм: 'о ' 0.0214, 'а ' 0.0180, 'п ' 0.0170, 'в ' 0.0163, 'и ' 0.0162
H2 (неперетинні) = 3.9832 біт [біграм: 356041, різних: 737]
 топ-5 біграм: 'о ' 0.0213, 'а ' 0.0179, 'п ' 0.0171, 'в ' 0.0163, 'и ' 0.0161

=== no_spaces ===
довжина: 598119, різних символів: 32
H1 = 4.4531 біт
 топ-5 символів: 'о' 0.1106, 'а' 0.0864, 'е' 0.0813, 'и' 0.0684, 'н' 0.0641
H2 (перетинні) = 4.1478 біт [біграм: 598118, різних: 851]
 топ-5 біграм: 'то' 0.0162, 'но' 0.0118, 'ст' 0.0112, 'на' 0.0110, 'по' 0.0109
H2 (неперетинні) = 4.1469 біт [біграм: 299059, різних: 827]
 топ-5 біграм: 'то' 0.0164, 'но' 0.0116, 'ст' 0.0112, 'по' 0.0111, 'на' 0.0111
(base) PS C:\Users\WarningKids\Desktop\Учеба\3 курс\5 семестр\Крипта\1 лаба>

```

СИМВОЛ	КІЛЬКІСТЬ	ЧАСТОТА
ТО	9704	0.016224
НО	7071	0.011822
СТ	6689	0.011183
НА	6560	0.010968

по	6522	0.010904
ов	6474	0.010824
ал	6391	0.010685
не	6289	0.010515
ко	6135	0.010257
ро	5709	0.009545
ла	5508	0.009209
ен	5442	0.009099
го	5434	0.009085
ра	5387	0.009007
ос	5374	0.008985
от	5345	0.008936
он	5306	0.008871
ни	5188	0.008674
ка	5089	0.008508
ер	4901	0.008194

Фрагмент таблиці частот перетинних біграм (для тексту з пробілами).

символ	кількість	частота
	113963	0.160042
о	66178	0.092936
а	51671	0.072563
е	48625	0.068286
и	40938	0.057491
н	38352	0.053859
т	36066	0.050649
л	31427	0.044134
с	30318	0.042577
р	28391	0.039870
в	28168	0.039557
к	21878	0.030724
у	17976	0.025244
м	17953	0.025212

п	17004	0.023879
д	16812	0.023610
г	11457	0.016089
я	11405	0.016016
з	10632	0.014931
ь	10626	0.014922

Фрагмент таблиці частот символів (для тексту з пробілами).

Для тексту з пробілами: $H_2(\text{перетинні}) = 3,9833$ біт, $H_2(\text{неперетинні}) = 3,9832$ біт — різниця лише в четвертому знаку після коми.

Для тексту без пробілів: $H_2(\text{перетинні}) = 4,1478$ біт, $H_2(\text{неперетинні}) = 4,1469$ біт — різниця також мінімальна.

Спосіб перетинних біграм використовує зсув на один символ, тому кожна буква тексту бере участь одразу у двох сусідніх парах, і кількість отриманих біграм майже дорівнює довжині тексту (712 081 для тексту з пробілами). Спосіб неперетинних біграм використовує зсув на два символи, тому кожна буква бере участь лише в одній парі, а кількість біграм удвічі менша (356 041). Через це неперетинний спосіб втрачає частину пар символів — ті, що починаються з непарної позиції тексту, — і статистика будується на вдвічі меншому обсязі спостережень.

Попри це, отримані значення H_2 практично збігаються. Це свідчить про те, що обсяг використаного тексту (712 082 символи) достатній для стійкої оцінки частот навіть при вдвічі меншій вибірці: втрата половини біграм не призводить до суттєвого зміщення результату.

Більше статистичних даних використовує спосіб з біграмами, що перетинаються, оскільки він враховує кожне сусідство символів у тексті, а не лише кожне друге. Саме тому в подальших розрахунках доцільно використовувати саме перетинні біграми як основну оцінку H_2 .

3. Дослідження впливу розподілу символів на ентропію

```

1 import random
2 from collections import Counter
3 from math import log2
4 from pathlib import Path
5
6 N = 100_000 # довжина кожної послідовності
7
8
9 def entropy(counts: Counter) -> float:
10     total = sum(counts.values())
11     return -sum((c / total) * log2(c / total) for c in counts.values())
12
13
14 def h1(text: str) -> float:
15     return entropy(Counter(text))
16
17
18 def make_a(source: str, n: int) -> str:
19     """А - фрагмент природного тексту (беремо перші n символів)."""
20     return source[:n]
21
22
23 def make_b(symbol: str, n: int) -> str:
24     """Б - повторення одного символу."""
25     return symbol * n
26
27
28 def make_v(alphabet: str, n: int) -> str:
29     """В - випадкова рівномірна послідовність з того самого алфавіту."""
30     return "".join(random.choice(alphabet) for _ in range(n))
31
32
33 def main() -> None:
34     random.seed(42) # для відтворюваності результатів
35
36     source = Path("text_no_spaces.txt").read_text(encoding="utf-8")
37     alphabet = sorted(set(source))
38
39     a = make_a(source, N)
40     b = make_b("А", N)
41     v = make_v(alphabet, N)

```

```

42
43     print(f"алфавіт (m={len(alphabet)}): {''.join(alphabet)}")
44     print(f"H0 = log2({len(alphabet)}) = {log2(len(alphabet)):.4f}\n")
45
46     for name, seq in (("А (природний текст)", a),
47                       ("Б (повторення символу)", b),
48                       ("В (випадкова рівномірна)", v)):
49         print(f"{name}: довжина={len(seq)}, різних символів={len(set(seq))}, "
50               f"H1={h1(seq):.4f} bit")
51
52
53 if __name__ == "__main__":
54     main()

```

Прогноз (до виконання розрахунків). Очікуваний порядок зростання H_1 : $B < A < V$.

- Для Б очікується $H_1 = 0$, оскільки послідовність складається з одного символу — розподіл вироджений, невизначеності немає.
- Для В очікується H_1 близько до максимального значення $H_0 = \log_2(m)$, оскільки всі символи алфавіту генеруються з однаковою ймовірністю — жоден символ не легше вгадати за інший.
- Для А очікується проміжне значення, оскільки реальний текст має нерівномірний, але не вироджений розподіл частот літер (одні літери зустрічаються значно частіше за інші).

```

(base) PS C:\Users\WarningKids\Desktop\Учеба\3 курс\5 семестр\Крипта\1 лаба> python part3.py
алфавіт (m=32): абвгдежзийклмнопрстуфхцщщрььэюя
H0 = log2(32) = 5.0000

A (природний текст): довжина=100000, різних символів=32, H1=4.4484 біт
Б (повторення символу): довжина=100000, різних символів=1, H1=-0.0000 біт
В (випадкова рівномірна): довжина=100000, різних символів=32, H1=4.9998 біт
(base) PS C:\Users\WarningKids\Desktop\Учеба\3 курс\5 семестр\Крипта\1 лаба> 

```

Для послідовності Б ймовірність єдиного символу дорівнює 1, усіх інших — 0. У формулі ентропії доданок $p \cdot \log_2(p)$ при $p=1$ дорівнює 0 ($\log_2(1)=0$), тому $H_1 = 0$. Це відповідає інтуїтивному розумінню ентропії як міри невизначеності: якщо результат відомий наперед, невизначеності немає.

Для послідовності В усі 32 символи рівноймовірні (кожен генерується з ймовірністю $1/32$), тому жоден символ не є більш очікуваним за інший — це найскладніший випадок для вгадування. Отримане значення 4,9998 практично збігається з теоретичним максимумом $H_0 = \log_2(32) = 5,0000$; невелике відхилення пояснюється статистичними коливаннями при скінченній довжині вибірки (100 000 символів).

Для послідовності А отримано $H_1 = 4,4484$ — проміжне значення між 0 і H_0 . Це відображає природу реального мовного джерела: частоти літер нерівномірні (наприклад, «о», «а», «е» зустрічаються значно частіше за «щ», «ъ», «ф»), але при цьому жодна літера не домінує повністю — використовуються всі 32 символи алфавіту.

Значення ентропії H_1 напряму залежить від форми розподілу ймовірностей символів джерела: чим більш «зосередженим» на окремих символах є розподіл, тим менша ентропія (менша невизначеність), і навпаки — чим ближче розподіл до рівноймовірного, тим ентропія вища. Природна мова займає проміжне положення між двома крайнощами саме завдяки нерівномірній, але не виродженій частотності літер, що й забезпечує можливість подальшого стиснення тексту (надлишковість мови).

4. Дослідження залежностей між символами

```

1 import random
2 from collections import Counter
3 from math import log2
4
5 N = 100_000 # довжина кожної послідовності (парна, щоб symbols were 50/50)
6 ALPHABET = "ab" # умовний двосимвольний алфавіт для наочності
7
8
9 def entropy(counts: Counter) -> float:
10     total = sum(counts.values())
11     return -sum((c / total) * log2(c / total) for c in counts.values())
12
13
14 def h1(text: str) -> float:
15     return entropy(Counter(text))
16
17
18 def bigram_counts(text: str, overlapping: bool) -> Counter:
19     step = 1 if overlapping else 2
20     return Counter(text[i:i + 2] for i in range(0, len(text) - 1, step))
21
22
23 def h2(text: str, overlapping: bool) -> float:
24     return entropy(bigram_counts(text, overlapping)) / 2
25
26
27 def make_g(alphabet: str, n: int) -> str:
28     """Г - символи в випадковому порядку, частоти рівні (по n/len(alphabet))."""
29     symbols = list(alphabet) * (n // len(alphabet))
30     random.shuffle(symbols)
31     return "".join(symbols)
32
33
34 def make_d(alphabet: str, n: int) -> str:
35     """Д - періодична структура: символи алфавіту по черзі, наприклад abab..."""
36     return (alphabet * (n // len(alphabet) + 1))[:n]

```

```

39 def main() -> None:
40     random.seed(42)
41
42     g = make_g(ALPHABET, N)
43     d = make_d(ALPHABET, N)
44
45     for name, seq in (("Г (випадковий порядок)", g),
46                      ("Д (періодична структура)", d)):
47         print(f"n=== {name} ===")
48         print(f"довжина={len(seq)}, частоти: {dict(Counter(seq))}")
49         print(f"h1 = {h1(seq):.4f} біт")
50         for overlapping, label in ((True, "перетинні"), (False, "неперетинні")):
51             bc = bigram_counts(seq, overlapping)
52             print(f"h2 ({label}) = {h2(seq, overlapping):.4f} біт "
53                   f"[біграми: {dict(bc.most_common())}]")
54
55
56 if __name__ == "__main__":
57     main()

```

```

(base) PS C:\Users\WarningKids\Desktop\Учеба\3 курс\5 семестр\Крипта\1 лаба> python part4.py

=== Г (випадковий порядок) ===
довжина=100000, частоти: {'b': 50000, 'a': 50000}
h1 = 1.0000 біт
h2 (перетинні) = 1.0000 біт [біграми: {'ba': 25016, 'ab': 25015, 'bb': 24984, 'aa': 24984}]
h2 (неперетинні) = 1.0000 біт [біграми: {'ab': 12618, 'ba': 12484, 'bb': 12449, 'aa': 12449}]

=== Д (періодична структура) ===
довжина=100000, частоти: {'a': 50000, 'b': 50000}
h1 = 1.0000 біт
h2 (перетинні) = 0.5000 біт [біграми: {'ab': 50000, 'ba': 49999}]
h2 (неперетинні) = -0.0000 біт [біграми: {'ab': 50000}]
(base) PS C:\Users\WarningKids\Desktop\Учеба\3 курс\5 семестр\Крипта\1 лаба>

```

Прогноз (до виконання розрахунків).

1. H_1 для Г та Д не повинні суттєво відрізнятися, оскільки обидві послідовності мають однакові частоти символів (по 50 000 'a' та 'b'), а H_1 враховує лише частоти, не порядок символів.
2. H_2 для Г та Д повинні суттєво відрізнятися, оскільки Д має жорстку залежність між сусідніми символами (після 'a' завжди йде 'b'), а Г такої залежності не має.
3. Менше значення H_2 очікується для Д, оскільки її структура повністю передбачувана.
4. Для Д значення H_2 (перетинні) та H_2 (неперетинні) також очікуються різними, оскільки регулярна структура може по-різному «потрапляти» у вікно біграми залежно від зсуву.

Чому тексти з однаковими частотами символів можуть мати однакове H_1 .

Значення H_1 обчислюється виключно за частотами окремих символів, без урахування їхнього порядку в послідовності. Оскільки Г і Д мають однакові частоти ('a' та 'b' по 50%), формула $H_1 = -\sum p \cdot \log_2(p)$ дає однаковий результат для обох — 1,0000 біт, що відповідає максимальній ентропії для двосимвольного алфавіту.

Чому їхні значення H_2 можуть відрізнятися.

H_2 враховує частоти пар сусідніх символів, тобто залежність між ними. У Г символи йдуть у випадковому порядку, тому всі чотири біграми ('aa', 'ab', 'ba', 'bb') зустрічаються приблизно однаково часто (~25%), і знання попереднього символу не допомагає передбачити наступний — $H_2 = H_1 = 1,0000$. У Д структура жорстко регулярна: після 'a' завжди йде 'b', тому серед можливих біграм реально зустрічаються практично лише 'ab' та 'ba' — знання попереднього символу майже повністю визначає наступний, тому H_2 суттєво нижче.

Чому для Д перетинні й неперетинні H_2 сильно відрізняються.

При зсуві на 1 символ (перетинні) вікно біграми послідовно «потрапляє» то у фазу 'ab', то у фазу 'ba', тому виникають дві різні біграми з імовірністю близько 50% кожна, звідки $H_2 = 0,5000$. При зсуві на 2 символи (неперетинні), починаючи з першого символу, вікно завжди потрапляє в одну й ту саму фазу, і виходить лише одна унікальна біграма 'ab', що повторюється завжди — невизначеності немає, $H_2 = 0,0000$. Це протилежність до природного тексту (розділ 2), де обидва способи давали майже однаковий результат: там текст достатньо «шумний», а тут структура строго регулярна, тому спосіб нарізки на біграми стає критично важливим.

Яку інформацію дозволяє врахувати модель біграм порівняно з моделлю незалежних символів.

Модель незалежних символів (H1) фіксує лише те, які символи «зустрічаються частіше», але ігнорує будь-які закономірності в їхньому розташуванні. Модель біграм (H2) додатково враховує, які символи йдуть один за одним, тобто «уловлює» структурні залежності в тексті. Саме тому дві послідовності з однаковими частотами символів, але різним порядком (Г і Д), можуть мати однакове H1, але кардинально різне H2 — це наочно доводить, що H1 недостатньо для повної характеристики джерела з пам'яттю.

Отже, порівняння Г і Д демонструє, що передбачуваність тексту визначається не лише частотами окремих символів, а й залежностями між ними. Моделі вищого порядку (H2 і вище) враховують дедалі довший контекст і тому дають дедалі точнішу (як правило, нижчу) оцінку реальної ентропії джерела.

5. Експериментальна оцінка ентропії джерела

Лабораторная работа №1

Произвольная часть текста:
ганизмизм

Использованные буквы:

Порядок n-граммы:
5 символов

Введенный символ:

Символ по счету:

Номер эксперимента: 50

Поле ввода символов:

Продолжить Другой

Неравенство для энтропии:
 $5.20113962332525 < H < 4.33452502874377$

Двоичная таблица угаданных символов:

Вероятности:

q[1]	= 0
q[2]	= 0
q[3]	= 0.0204081
q[4]	= 0.0612244
q[5]	= 0.1020408
q[6]	= 0.0612244
q[7]	= 0.0204081
q[8]	= 0.0204081
q[9]	= 0
q[10]	= 0.020408
q[11]	= 0
q[12]	= 0
q[13]	= 0.020408
q[14]	= 0.040816
q[15]	= 0.040816
q[16]	= 0
q[17]	= 0.020408
q[18]	= 0.102040
q[19]	= 0.040816
q[20]	= 0.040816
q[21]	= 0
q[22]	= 0
q[23]	= 0.040816
q[24]	= 0.040816
q[25]	= 0.040816
q[26]	= 0.040816
q[27]	= 0.081632
q[28]	= 0.040816
q[29]	= 0
q[30]	= 0.020408
q[31]	= 0.040816
q[32]	= 0.040816

Строка состояния:

Лабораторная работа №1

Произвольная часть текста:
ит_е_том_что_мы_вер

Использованные буквы:

Порядок n-граммы:
15 символов

Введенный символ:

Символ по счету:

Номер эксперимента: 50

Поле ввода символов:

Продолжить Другой

Неравенство для энтропии:
 $3.36044179724533 < H < 3.69924338674848$

Двоичная таблица угаданных символов:

Вероятности:

q[1]	= 0.2857142
q[2]	= 0.0816326
q[3]	= 0.0408163
q[4]	= 0.0408163
q[5]	= 0.0408163
q[6]	= 0.0612244
q[7]	= 0
q[8]	= 0
q[9]	= 0.0204081
q[10]	= 0
q[11]	= 0
q[12]	= 0.020408
q[13]	= 0
q[14]	= 0.040816
q[15]	= 0.020408
q[16]	= 0.040816
q[17]	= 0.040816
q[18]	= 0.020408
q[19]	= 0
q[20]	= 0.020408
q[21]	= 0.040816
q[22]	= 0
q[23]	= 0.020408
q[24]	= 0
q[25]	= 0.020408
q[26]	= 0.020408
q[27]	= 0.040816
q[28]	= 0.020408
q[29]	= 0
q[30]	= 0
q[31]	= 0.020408
q[32]	= 0.040816

Строка состояния:

Лабораторная работа №1

Произвольная часть текста:
го_мнения_держались_люди_и_по

Использованные буквы:

Порядок n-граммы:
☐ 5 символов
☐ 10 символов
☐ 15 символов
☐ 20 символов
☐ 25 символов
☒ 30 символов
☐ 35 символов
☐ 40 символов
☐ 45 символов
☐ 50 символов

Введенный символ:

Символ по счету:

Номер эксперимента: 50

Неравенство для энтропии:
 $2,91557554497398 < H < 3,53688254762286$

Двоичная таблица угаданных символов:

10000000000000000000000000000000
10000000000000000000000000000000
10000000000000000000000000000000
10000000000000000000000000000000
10000000000000000000000000000000
10000000000000000000000000000000

Поле ввода символов:

Продолжить Другой

Вероятности:

q[1] = 0,3673469
q[2] = 0,0612244
q[3] = 0,0204081
q[4] = 0,0408163
q[5] = 0,0408163
q[6] = 0,0408163
q[7] = 0,0408163
q[8] = 0,0408163
q[9] = 0,0408163
q[10] = 0
q[11] = 0,020408
q[12] = 0
q[13] = 0,020408
q[14] = 0
q[15] = 0,020408
q[16] = 0,020408
q[17] = 0,020408
q[18] = 0,020408
q[19] = 0,020408
q[20] = 0
q[21] = 0,020408
q[22] = 0
q[23] = 0
q[24] = 0
q[25] = 0
q[26] = 0,081632
q[27] = 0
q[28] = 0
q[29] = 0,040816
q[30] = 0
q[31] = 0
q[32] = 0,020408

Строка состояния:

Оцінка	Отримане значення
H_1	4,4484
H_2	3,9833
$H^{(10)}$	$5,2011 < H < 4,3345$
$H^{(20)}$	$3,3604 < H < 3,8992$
$H^{(30)}$	$2,9156 < H < 3,5369$

Аналіз результатів:

1. Чому наведені оцінки ентропії не повинні обов'язково збігатися?

Кожна модель враховує різну глибину контексту (від 0 до 30 символів). Крім того, оцінки $H^{(n)}$ засновані на вгадуванні людиною, що дає суб'єктивний результат і містить похибку порівняно з точним комп'ютерним підрахунком H_1 та H_2 .

2. Які статистичні залежності враховуються при визначенні H_1 та H_2 ?

H_1 враховує лише те, як часто окрема літера зустрічається в тексті, без прив'язки до сусідів. H_2 вже враховує зв'язок між двома сусідніми літерами (ймовірність їхньої сумісної появи).

3. Чому врахування довших залежностей між символами може змінювати оцінку ентропії природної мови?

Природна мова будується за правилами граматики та синтаксису. Бачачи частину слова чи речення, ми вже здогадуємося, яка літера буде наступною, тому міра невизначеності (ентропія) падає.

4. Яку тенденцію можна спостерігати при переході від H_1 до оцінок, що враховують дедалі довший контекст?

Чим довший контекст ми знаємо, тим меншим стає значення ентропії тексту, оскільки символи стають більш передбачуваними.

6. Оцінювання надлишковості джерела

Для російського алфавіту з 32 літер максимальна ентропія складає $H_0 = \log_2(32) = 5$ біт. Розрахуємо надлишковість за формулою $R = 1 - (H / H_0)$:

- Для незалежних символів: $R_1 = 1 - (4,4484 / 5) = \mathbf{0,1103 (11,03\%)}$
- Для біграм: $R_2 = 1 - (3,9833 / 5) = \mathbf{0,2033 (20,33\%)}$
- Для контексту 10 символів: $R_{10} = 1 - (4,3345 / 5) = \mathbf{0,1331 (13,31\%)}$
- Для контексту 20 символів: $R_{20} = 1 - (3,8992 / 5) = \mathbf{0,2202 (22,02\%)}$
- Для контексту 30 символів (беремо верхню межу): $R_{30} = 1 - (3,5369 / 5) = \mathbf{0,2926 (29,26\%)}$

Аналіз:

Проаналізувавши отримані дані, ми побачили, що при врахуванні контексту надлишковість загалом зростає (від 11,03% для H_1 до 29,26% для H_{30}). Також ми звернули увагу на цікавий нюанс: розрахована нами надлишковість R_{10} (13,31%) виявилася меншою за R_2 (20,33%). Ми пояснюємо це тим, що H_2 було обчислено скриптом абсолютно точно на всьому масиві тексту, тоді як $H(10)$ — це результат моїх власних 50 спроб ручного вгадування у програмі, а людина об'єктивно не здатна ідеально врахувати всю статистику мови. Проте, коли я збільшував контекст до 20 та 30 символів, мені ставало значно простіше вгадувати літери. Через це отримана ентропія падала, а надлишковість стрімко зростала. Наші експерименти практично доводять, що природна мова має значну надлишковість, і чим більш передбачуваним для нас є текст, тим вища його надлишковість.

Аналіз результатів (відповіді на запитання)

1. Чому природний текст має меншу ентропію, ніж випадкова послідовність?

У випадковому тексті всі букви випадають однаково часто, а в природному є часті літери (голосні) і дуже рідкісні. У нашому третьому експерименті H_1 природного тексту склало 4,44 біт, а випадкового — 4,99 біт, що майже дорівнює максимуму.

2. Що враховує H_2 , але не враховує H_1 ?

H_2 враховує статистичну залежність літери від попередньої, тоді як H_1 розглядає кожну літеру ізольовано від інших.

3. Чи можуть тексти мати однакове H_1 , але різне H_2 ?

Так. У 4-му експерименті випадкова послідовність та періодична (ababab) мали однакове $H_1 = 1$ біт, бо частоти символів були рівні (по 50%). Але H_2 у періодичної впало до 0,5 біт через жорстку передбачуваність, а у випадкової залишилось 1 біт.

4. Як вилучення пробілів вплинуло на ентропію?

Ентропія зросла. За нашими розрахунками, H_1 збільшилось з 4,37 до 4,45 біт. Пробіл був найчастішим символом. Коли його прибрали, частоти інших букв стали більш рівномірними, тому загальна невизначеність зросла.

5. Чому оцінки H_2 (перетинні/неперетинні) відрізняються?

Спосіб без перетину бере лише кожну другу пару, втрачаючи половину зв'язків. Для звичайного тексту різниця вийшла мізерною, але для послідовності "ababab" неперетинний спосіб дав $H_2 = 0$, бо вікно зміщувалося так, що завжди захоплювало лише одну пару "ab".

6. Чому всі ці значення відрізняються?

Тому що ці моделі враховують різну довжину попереднього контексту (пам'ять джерела). Що більший шматок тексту ми вже прочитали, то легше вгадати наступну літеру.

7. Який експеримент найкраще показує залежності між символами?

Експеримент 4 (порівняння випадкового тексту і періодичного "ababab"). Він показав, що навіть при однакових частотах літер, сама лише зміна їхнього порядку кардинально знижує ентропію H_2 .

8. Як це пов'язано зі стисненням тексту?

Ентропія показує теоретичну межу, до якої можна стиснути дані. Наша розрахована надлишковість склала близько 29% — це означає, що як мінімум третину об'єму природного тексту можна стиснути архіваторами без втрати інформації.

Загальні висновки

Під час виконання комп'ютерного практикуму було практично засвоєно поняття ентропії та надлишковості тексту. Експерименти та програмні розрахунки підтвердили теоретичні дані: природна мова має меншу ентропію, ніж випадкова послідовність символів, завдяки нерівномірній частоті літер та правилам мови. Дослідження довели, що врахування контексту (залежностей у біграмах або попередніх символах) суттєво зменшує міру невизначеності. Розрахована надлишковість тексту доводить принципову можливість його стиснення без втрати даних.